

# M11.5.5 — Strategic Alignment Review

## Part 1 — Client Briefing

**Project:** FH6 Farm Tool

**Review Type:** Founder / Lead Developer Strategic Briefing

**Status Date:** Post-M11.5 (Operator Runbooks Complete)

**Purpose:** Honest assessment of current reality, launch maturity, risks, and direction.

---

## 1. Executive Summary

The FH6 Farm Tool has progressed unusually quickly.

Within approximately one week, the project has evolved from concept and isolated macro experimentation into a structured, modular, documented software system with guarded automation, validation boundaries, GitHub version control, milestone discipline, and operational documentation.

Current status is:

### **Controlled MVP Ready — Developer/Manual Use**

This means:

- The software has meaningful functionality.
- Core systems are validated within constrained boundaries.
- The architecture is increasingly stable.
- Documentation maturity is above average for an early-stage project.
- Operator trust and safety philosophy are visible in implementation.

This does **not** mean:

- public release readiness
- unattended reliability
- mass-user readiness
- production-grade robustness
- broad feature completeness

The project is in a healthy state, but remains:

### **an intentionally constrained premium MVP in hardening**

not:

“finished software.”

---

## 2. Current Project State

Current development stage:

**MVP Hardening & Polish**

The project has intentionally transitioned away from:

feature acquisition

toward:

reliability, documentation, trust, and operational maturity.

This transition is correct.

At the current maturity stage, adding substantial new functionality too early would likely increase fragility and reduce launch confidence.

The software is now sufficiently developed that:

the biggest risk is no longer “can we build it?”

but instead:

“can we preserve quality while finishing it?”

This is a healthier problem.

---

## 3. What Currently Exists

### Auto1 — Race Automation

Status:

**Validated**

Capabilities:

- guarded/manual execution
- finite cycles
- profile-driven timing
- F8 stop support
- controlled restart loop
- reliability validation completed

Current maturity:

Stable within validated assumptions.

Confidence level:

High.

---

## Auto2 — Buy Car Automation

Status:

**Validated (Controlled Scope)**

Capabilities:

- guarded/manual execution
- menu traversal validation
- one-car purchase validation
- profile-driven timing
- confirmation-gated purchase path
- F8 stop support

Important limitation:

Auto2 is not yet positioned as a broad production purchasing tool.

Current maturity:

Reliable within constrained assumptions.

Confidence level:

Medium-High.

Reason:

Spending risk and FH6 menu dependency increase fragility relative to Auto1.

---

## Auto3 — Skill Tree Automation

Status:

**Validated (Controlled Boundary)**

Capabilities:

- first-car exception flow
- later-car recovery flow
- guarded/manual unlock path
- test-mode real-input validation
- multi-car traversal validation
- F8 stop support
- reset-to-grid baseline behavior

Current hard limits:

```
max cars = 4
start row = A
validated traversal:
A1 → B1 → C1 → A2
```

Current maturity:

Strong for a constrained scope.

Important reality:

Auto3 is impressive technically, but also the most fragile system because:

- FH6 assumptions matter heavily
- timing matters
- skill points may be spent
- menu alignment matters
- traversal correctness matters

Confidence level:

Medium.

Not because implementation is weak.

Because scope sensitivity is inherently higher.

---

## 4. What Is Actually Validated

This section intentionally separates:

“exists”

from

“validated reality.”

Validated today:

### **Auto1**

- guarded/manual race automation
- restart loop
- timing system
- F8 interruption
- finite execution

### **Auto2**

- menu navigation
- test-mode validation
- one-car purchase validation
- confirmation safety
- finite execution

### **Auto3**

- first-car path
- later-car path
- 12-second recovery
- grid traversal
- A-start flow
- 4-car validation
- reset-to-grid
- guarded unlock flow
- test-mode validation

Not validated:

- unattended execution
- mass-user variability
- different FH6 UI/layout changes
- long-duration reliability
- advanced Auto3 scaling
- Auto4
- UI workflows
- packaging/licensing systems

This distinction matters.

The project is stronger than a prototype.

But weaker than a public release.

That is normal.

## 5. What Is Frozen / Intentionally Deferred

Current frozen boundaries:

```
Auto3 > 4 cars  
Auto3 B/C row starts  
Auto4/remove-car  
unattended automation  
timing optimization  
broad feature expansion  
production Auto3 mode  
UI-first direction
```

These are frozen intentionally.

Reason:

The project philosophy prioritizes:

trust before capability.

Important distinction:

These are not abandoned.

They are:

intentionally deferred.

Deferred scope remains visible and revisit-able later.

---

## 6. Current Risks & Unknowns

### Risk 1 — Premature Complexity

The largest technical risk is:

expanding too early.

Examples:

- Auto4 too soon
- packaging complexity too soon
- UI too soon
- licensing too soon

- excessive automation flexibility too soon

Current recommendation:

maintain scope discipline.

---

## Risk 2 — Founder Perfectionism

A realistic risk exists that:

launch threshold moves endlessly.

Because quality standards are high.

This is both:

**strength**

and

**danger**

Without explicit launch criteria, risk becomes:

one more improvement  
one more polish pass  
one more feature

Recommendation:

define “good enough” before launch.

---

## Risk 3 — Documentation Freshness

Development pace has been unusually fast.

Current-state documents can become stale quickly.

Recommendation:

Treat operational docs as:

living snapshots

not permanent truth.

Update:

at milestone completion

—not every commit.

---

## Risk 4 — Real-World Variability

The biggest unknown remains:

real operator variability.

Future unknowns:

- different PCs
- input delays
- resolution differences
- FH6 updates
- human misuse
- misunderstanding of baseline assumptions

This is unavoidable.

Controlled beta later will be important.

---

## 7. Realistic Launch Interpretation

Current recommended launch definition:

**Controlled Premium MVP**

Meaning:

Launch should mean:

trustworthy  
safe  
conservative  
well-documented  
supervised  
premium-feeling  
predictable

Launch should **not** mean:



feature complete  
fully automated  
mass market  
"done"

This distinction is strategically important.

The likely correct launch philosophy is:

**small, premium, controlled, trust-first**

before scale.

---

## 8. Immediate Recommended Direction

Current recommendation:

Continue with:

M11.6 – Command Surface Hardening  
M11.7 – Reliability & Edge Case Audit  
M11.8 – Packaging Readiness Planning  
M12 – Controlled Internal Beta

Do **not** pivot into:

- website building
- marketing
- advanced branding
- pricing debates
- complex licensing

yet.

Business discussions should happen:

in proportion to development relevance.

---

## 9. Founder / Strategic Assessment

### Strengths

#### 1. Strong Product Judgment

You have shown unusually good instinct for:

what not to build.

This is uncommon.

Many founders struggle more with:

restraint

than ideas.

You have repeatedly prioritized:

trust  
clarity  
discipline  
safety  
scope control

This has materially improved the project.

---

#### 2. High Learning Velocity

Progress speed has been unusually high.

Not because of coding skill alone.

But because of:

decision quality.

You absorb structure quickly.

You challenge assumptions.

You refine process.

This matters more long-term than raw coding skill.

---

### 3. Good Founder-Developer Behavior

You generally avoid:

ego-driven decision making.

You challenge reasoning.

But you also revise positions when logic is stronger.

This is a meaningful strength.

---

## Weaknesses / Risks

### 1. Perfectionism Risk

The largest founder risk currently.

You care deeply about quality.

Good.

But there is risk of:

over-polishing before learning from launch.

Recommendation:

Remember:

launch teaches things internal development cannot.

---

### 2. Underestimating Operational Complexity

Potential future risk.

Areas to watch:

- website trust
- onboarding
- payments
- customer expectations
- support burden
- installer trust
- licensing complexity

You are fully capable of learning these.

But current experience level likely underestimates them.

Recommendation:

simplify aggressively.

Especially for v1.

---

### 3. High Ambition Risk

You naturally think structurally and expansively.

Strength.

But also risk.

There will be temptation to think:

while we're here...

That is dangerous late-stage.

Recommendation:

protect milestone discipline.

---

## Final Assessment

Current state is:

**meaningfully ahead of where most first software projects are after one week**

while still remaining:

grounded in reality.

This is not hype.

It is also not launch-ready.

But the trajectory is strong.

The current recommendation is:

stay disciplined, preserve quality, finish intentionally, launch conservatively.